

Experiment 3: Delays

Before Lab:

The circuit uses a 4MHz crystal. Calculate the expected instruction-cycle time before making measurements.

$$T_{osc} = 1/F_{osc}$$

$$T_{osc} = 1/4\text{MHz}$$

$$T_{osc} = 250\text{ns}$$

$$T_{cy} = 4 \cdot T_{osc}$$

$$T_{cy} = 4 \cdot 250\text{ns}$$

$$T_{cy} = 1\mu\text{s}$$

$$F_{osc} = 4/T_{cy}$$

$$F_{osc} = 4/1\mu\text{s}$$

$$F_{osc} = 4\text{MHz}$$

Determine the expected number of instruction cycles for:

- RBQ PW
- RBQ PS
- One complete RBQ Period

According to the code used for the counter (Located on Figure 2.8 on pg 12), the instructions should create a pulse width and pulse space of 5 μs , and an overall period of 10 μs .

In the Lab:

1.) Measure the Lab 2 Counter:

Build and program the PIC16F883 using your working Lab 2 PortB counter program. Connect the oscilloscope to RBQ, the least significant bit of the counter. Measure and Record:

- PW
- PS
- Period
- Frequency

Capture an oscilloscope image showing the waveform to verify the measurement. Compare PW and PS and explain why RBQ changes state at the observed rate based on the binary count and the instruction in the main loop.

On page 13, Figure 2.10 is the capture of RBQ on The PORTB counting circuit. The oscilloscope measured:

- PW = 5 μs
- PS = 5 μs
- Period = 10 μs
- Frequency = 100K Hz

The reason RBQ changes state at the observed rate based on the binary count is because each instruction is 1 to 2 t_{cy} , adding or removing instructions changes the time for the main cycle.

Experiment 3: Delays

In the Lab:

2.) Determine the instruction-cycle time:

Use the measured waveform and the instruction-cycle analysis of the main loop to calculate the actual instruction-cycle time. If N instruction cycles occur during one measured interval: $T_{cy} = t_{measured}/N$. If you use the complete waveform period, account for all instruction cycles executed during the entire period.

$$T_{cy} = t_{measured}/N$$

$$T_{cy} = 10\mu s / 10$$

$$T_{cy} = 1\mu s$$

3.) Determine the oscillator frequency:

Calculate the actual oscillator frequency from the measured instruction-cycle time:

$$F_{osc} = 1/T_{cy}$$

$$F_{osc} = 1/1\mu s$$

$$F_{osc} = 1\text{ MHz}$$

Compare the result with the nominal 4MHz crystal frequency. Calculate the percent difference:

$$\% \text{ diff} = (|F_{measured} - F_{nominal}| / F_{nominal}) \cdot 100$$

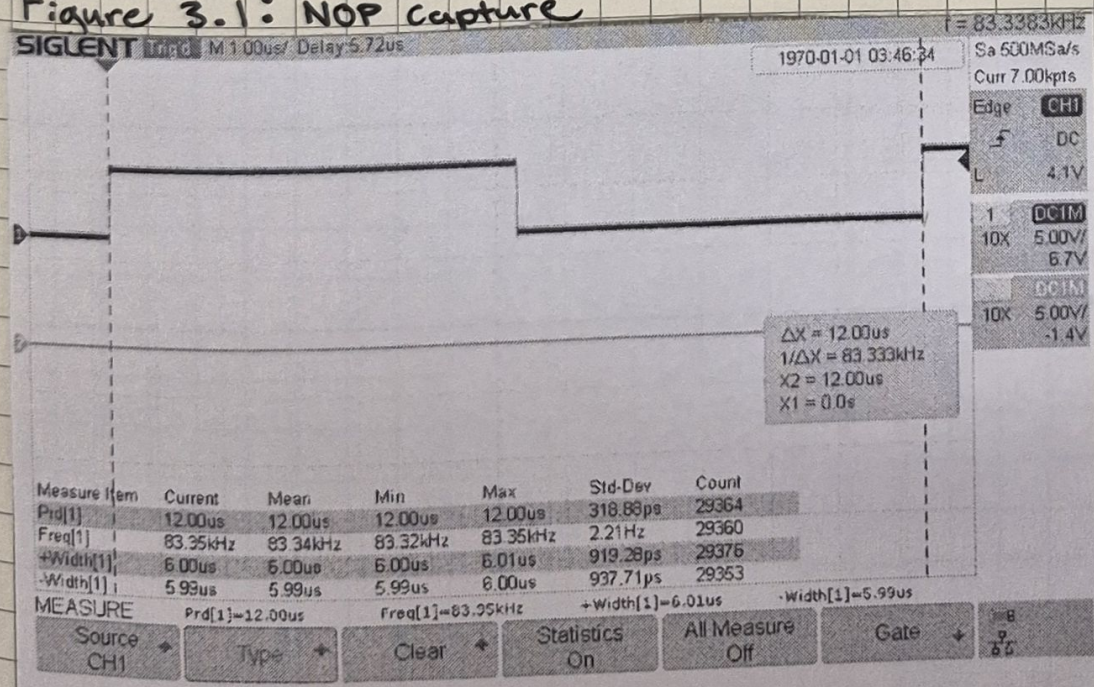
$$\% \text{ diff} = (|4\text{ MHz} - 1\text{ MHz}| / 1\text{ MHz}) \cdot 100$$

$$\% \text{ diff} = 0\%$$

4.) Add and remove NOP:

Add one NOP instruction to the repeating execution path of the main loop. Before programming the PIC, predict the resulting change in the PW, PS, period, and frequency.

Figure 3.1: NOP capture



Experiment 3: Delays

In the Lab:

Figure 3.2: Instruction set time and main code

TABLE 15-2: PIC16F882/883/884/886/887 INSTRUCTION SET

Mnemonic, Operands	Description	Cycles	14-Bit Opcode		Status Affected	Notes
			MSb	LSb		
BYTE-ORIENTED FILE REGISTER OPERATIONS						
ADDWF	f, d	Add W and f	1	00 0111 dfff ffff	C, DC, Z	1, 2
ANDWF	f, d	AND W with f	1	00 0101 dfff ffff	Z	1, 2
CLRF	f	Clear f	1	00 0001 1fff ffff	Z	2
CLRWF	—	Clear W	1	00 0001 0xxx xxxx	Z	
COMF	f, d	Complement f	1	00 1001 dfff ffff	Z	1, 2
DECF	f, d	Decrement f	1	00 0011 dfff ffff	Z	1, 2
DECFSZ	f, d	Decrement f, Skip if 0	1(2)	00 1011 dfff ffff		1, 2, 3
INCF	f, d	Increment f	1	00 1010 dfff ffff	Z	1, 2
INCFSZ	f, d	Increment f, Skip if 0	1(2)	00 1111 dfff ffff		1, 2, 3
IORWF	f, d	Inclusive OR W with f	1	00 0100 dfff ffff	Z	1, 2
MOVF	f, d	Move f	1	00 1000 dfff ffff	Z	1, 2
MOVWF	f	Move W to f	1	00 0000 1fff ffff		
NOP	—	No Operation	1	00 0000 0xx0 0000		
RLF	f, d	Rotate Left f through Carry	1	00 1101 dfff ffff	C	1, 2
RRF	f, d	Rotate Right f through Carry	1	00 1100 dfff ffff	C	1, 2
SUBWF	f, d	Subtract W from f	1	00 0010 dfff ffff	C, DC, Z	1, 2
SWAPF	f, d	Swap nibbles in f	1	00 1110 dfff ffff		1, 2
XORWF	f, d	Exclusive OR W with f	1	00 0110 dfff ffff	Z	1, 2
BIT-ORIENTED FILE REGISTER OPERATIONS						
BCF	f, b	Bit Clear f	1	01 00bb bfff ffff		1, 2
BSF	f, b	Bit Set f	1	01 01bb bfff ffff		1, 2
BTFSC	f, b	Bit Test f, Skip if Clear	1 (2)	01 10bb bfff ffff		3
BTFSS	f, b	Bit Test f, Skip if Set	1 (2)	01 11bb bfff ffff		3
LITERAL AND CONTROL OPERATIONS						
ADDLW	k	Add literal and W	1	11 111x kkkk kkkk	C, DC, Z	
ANDLW	k	AND literal with W	1	11 1001 kkkk kkkk	Z	
CALL	k	Call Subroutine	2	10 0kkk kkkk kkkk		
CLRWDT	—	Clear Watchdog Timer	1	00 0000 0110 0100	TO, PD	
GOTO	k	Go to address	2	10 1kkk kkkk kkkk		
IORLW	k	Inclusive OR literal with W	1	11 1000 kkkk kkkk	Z	
MOVLW	k	Move literal to W	1	11 00xx kkkk kkkk		
RETFIE	—	Return from interrupt	2	00 0000 0000 1001		
RETLW	k	Return with literal in W	2	11 01xx kkkk kkkk		
RETURN	—	Return from Subroutine	2	00 0000 0000 1000		
SLEEP	—	Go into Standby mode	1	00 0000 0110 0011	TO, PD	
SUBLW	k	Subtract k from literal	1	11 110x kkkk kkkk	C, DC, Z	
XORLW	k	Exclusive OR literal with W	1	11 1010 kkkk kkkk	Z	

Main:

```
// main code goes here
BCF 0x03,6 ; Sets Bank 0
BCF 0x03,5 ; Sets Bank 0
INCF 0x06,1 ; Increment PORTB by 1
NOP
goto Main
```

End

Compare the measured timing change with the instruction-cycle time determined earlier. Remove the NOP, program the PIC and verify that the waveform returns to it's previous timing.

After removing the NOP, the timing returned to the 10 μ S period.

Experiment 3: Delays

Part 2: Create a Precise Short Delay

Goal:

Create a new PIC16F883 project that toggles RB0 using XORWF, then add a calculated software delay to ^{HLC} produce a 10kHz square wave. Use a nominal cycle of: $T_{cy} = 1\mu s$ for all initial predictions and design calculations. After measuring the circuit, use the actual instruction-cycle time determined in part 1 to calculate the expected real timing and compare it with the measured results.

Before Lab:

Create a New MPLab X Project for this part. Use the project structure, PIC configuration, oscillator configuration, and assembly conventions established in Lab 2. Reference earlier documentation where nothing changed. Begin with this repeating output loop:

Main:

```
MOVLW 0x01    ; 1tcy
XORWF PORTB,1 ; 1tcy
GOTO Main     ; 2tcy
```

Determine what waveform this program should produce on RB0. ($T_{cy} = 1\mu s$). Predict the PW, PS, Period, Frequency, and Duty cycle.

$$1+1+2 = 4 \text{ instruction cycles}$$

$$4(1\mu s) = 4\mu s$$

$$\boxed{PW \text{ and } PS = 4\mu s}$$

$$T = PW + PS$$

$$T = 4\mu s + 4\mu s$$

$$\boxed{T = 8\mu s}$$

$$F = 1/T$$

$$F = 1/8\mu s$$

$$\boxed{F = 125kHz}$$

$$DC = PW/T \cdot 100$$

$$DC = 4\mu s/8\mu s$$

$$\boxed{DC = 50\%}$$

In the Lab:

1.) Verify the XOR toggle loop:

Build and program the new project using the unmodified loop. Measure RB0 using the oscilloscope and frequency counter. Record the PW, PS, Period, frequency, and duty cycle.

- $PW = 4\mu s$
- $PS = 4\mu s$
- $Period = 8\mu s$
- $Frequency = 125kHz$
- $Duty Cycle = 50\%$

Capture can be found on the next page.

Experiment 3: Delays

In the Lab:

Compare the measured values with the prediction based on $T_{cy} = 1\mu s$. Then use the actual instruction-cycle time measured in Part 1 to calculate the expected PW, PS, period, frequency, and duty cycle for the actual processor. Compare:

- 1.) The nominal prediction using $1\mu s$ T_{cy}
- 2.) The expected result using the measured T_{cy}
- 3.) The Oscilloscope measurement
- 4.) The Frequency Counter

Explain why `XORWF PORTB, 1` ^{HLC} causes RBO to alternate between HIGH and LOW.

Figure 3.3: Measured XOR Toggle Loop

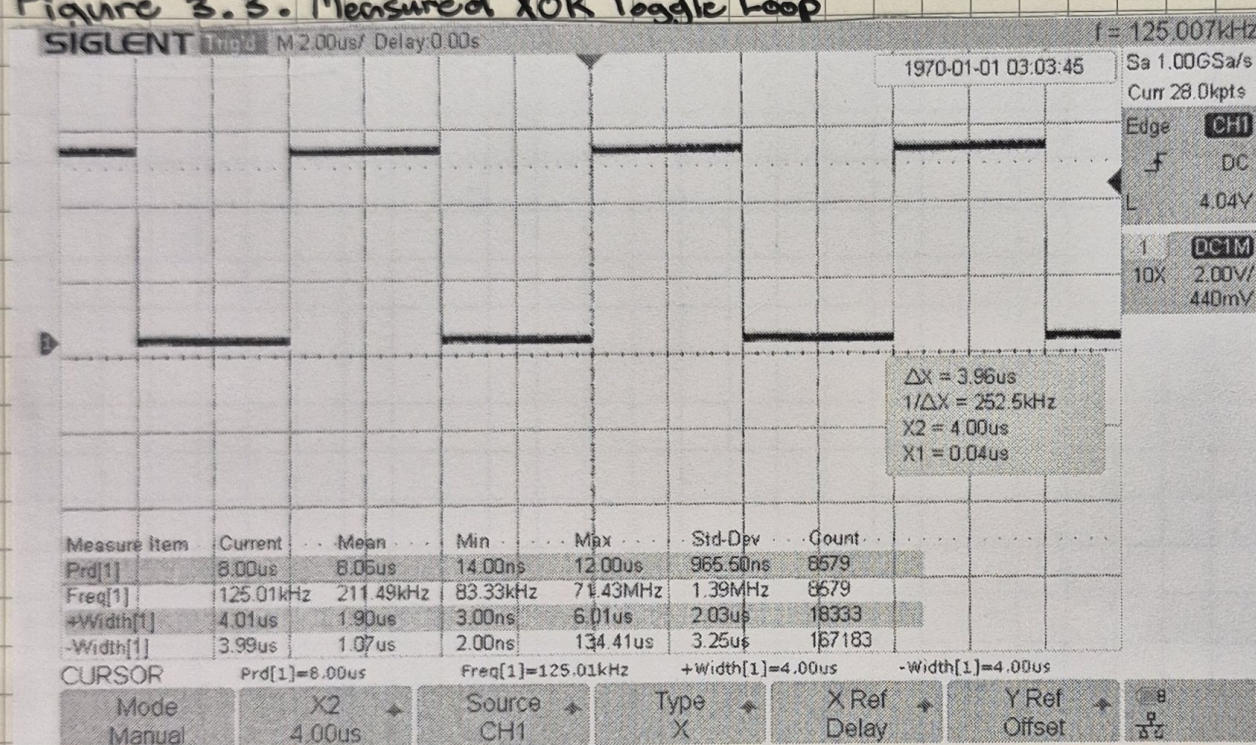


Table 3.1: Measured and predicted values

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	4μs	4μs	4μs	4.005 μs
PS	4μs	4μs	4μs	3.994 μs
Period	8μs	8μs	8μs	7.999 μs
Frequency	125k Hz	125k Hz	125k Hz	125.005k Hz
Duty cycle	50%	50%	50%	50 %

The `XORWF` compares `PORTB` with `W=0x01` and then stores the result back into `RBO`, so on every cycle it alternates `RBO` and `RBI` ^{HLC} and nothing else.

Experiment 3: Delays

In the Lab:

2.) Determine the required timing:

Modify the program so that RB0 produces:

- 10 KHz frequency
- 50% Duty Cycle

$$T = 1/10k = 100\mu s \rightarrow PW = PS = T/2 = 50\mu s \quad (T_{cy} = 1\mu s)$$

Determine the number of instruction cycles required between successive RB0 transitions. Account for every instruction executed between transitions. Do not treat the delay loop as if it were the only code consuming time.

To achieve the 50 μs PW, the code needs an additional 46 more instructions, which can be made either using NOPs, a delay loop, or additional code in the main.

3.) Create a software delay loop:

Use an 8-bit counter and DECFSZ to add the required delay. Use the following example to analyze how a software delay loop works:

Figure 3.4: Code Example for Delay

```
count EQU 0x20 ;use data memory address 0x20 as an 8-bit loop counter

MainLoop:
    MOVLW 0x01 ;1 Tcy code not part of delay
    XORWF PORTB,1 ;1 Tcy code not part of delay

    MOVLW 0x09 ;1 Tcy delay overhead
    MOVWF count ;1 Tcy delay overhead
DelayLoop:
    DECFSZ count,1 ;1(2) 1 Tcy if not zero, 2 Tcy if zero
    GOTO DelayLoop ;2 Tcy

    GOTO MainLoop ;2 Tcy code not part of delay
```

Work from the inside of the delay loop outwards.

Inner loop:

$$T_{delay} = N(1+2) - 1 + 1 + 1$$

$$T_{delay} = 3N + 1$$

Example $\downarrow$ $N=9$

$$T_{delay} = 3(9) + 1$$

$$T_{delay} = 28\mu s$$

The delay doesn't describe the complete PW or PS. The other instructions in Mainloop also consumes time!

Outer Loop:

$$PW = (3N + 1) + 1 + 1 + 2$$

$$PW = 3N + 5$$

Experiment 3: Delay

In the Lab:

4.) Determine the counter value:

Use $PW = PS = 50\mu s$ and $T_{cy} = 1\mu s$ to determine the required number of instruction cycles between RBQ transitions. Then determine:

- Instruction cycles consumed outside the delay
- Instruction cycles required from the delay
- The required counter value
- Whether any NOP instructions are needed to achieve the desired timing.

$$50\mu s = 3N + S$$

$$45\mu s = 3N$$

$$N = 15 \text{ cycles}$$

The code will not need a NOP added since the amount of cycles needed was a whole number.

Figure 3.5: Modified Code

```
Main:
    MOVLW 0x01      ;1 TCY code not part of delay
    XORWF PORTB,1   ;1 TCY code not part of delay

    MOVLW 0x0F      ;1 TCY delay overhead
    MOVWF count     ;1 TCY delay overhead

DelayLoop:
    DECFSZ count,1   ;1(2) 1 TCY if not zero, 2 TCY if zero
    GOTO DelayLoop   ;2 TCY
    GOTO Main        ;2 TCY code not part of delay
End
```

The highlighted portion of the code is the only part that changed since it's setting how many cycles the delay will run, setting the frequency desired.

^{HLC}
Table 3.2: ~~Measured Delay data~~

5.) Build and measure the calculated delay:

Build and program the PIC16F883 using the calculated delay. Measure RBQ using both the oscilloscope and frequency counter. Record:

- PW
- PS
- Period
- Frequency
- Duty Cycle

First compare measurements with the nominal prediction based on $T_{cy} = 1\mu s$. Then, use the actual T_{cy} measured in part 1 to calculate what PW, PS, period, frequency, and duty cycle the program should produce on the actual processor.

Refer to the next page for table and capture.

Experiment 3: Delay

In the Lab:

Figure 3.6: Delay Capture

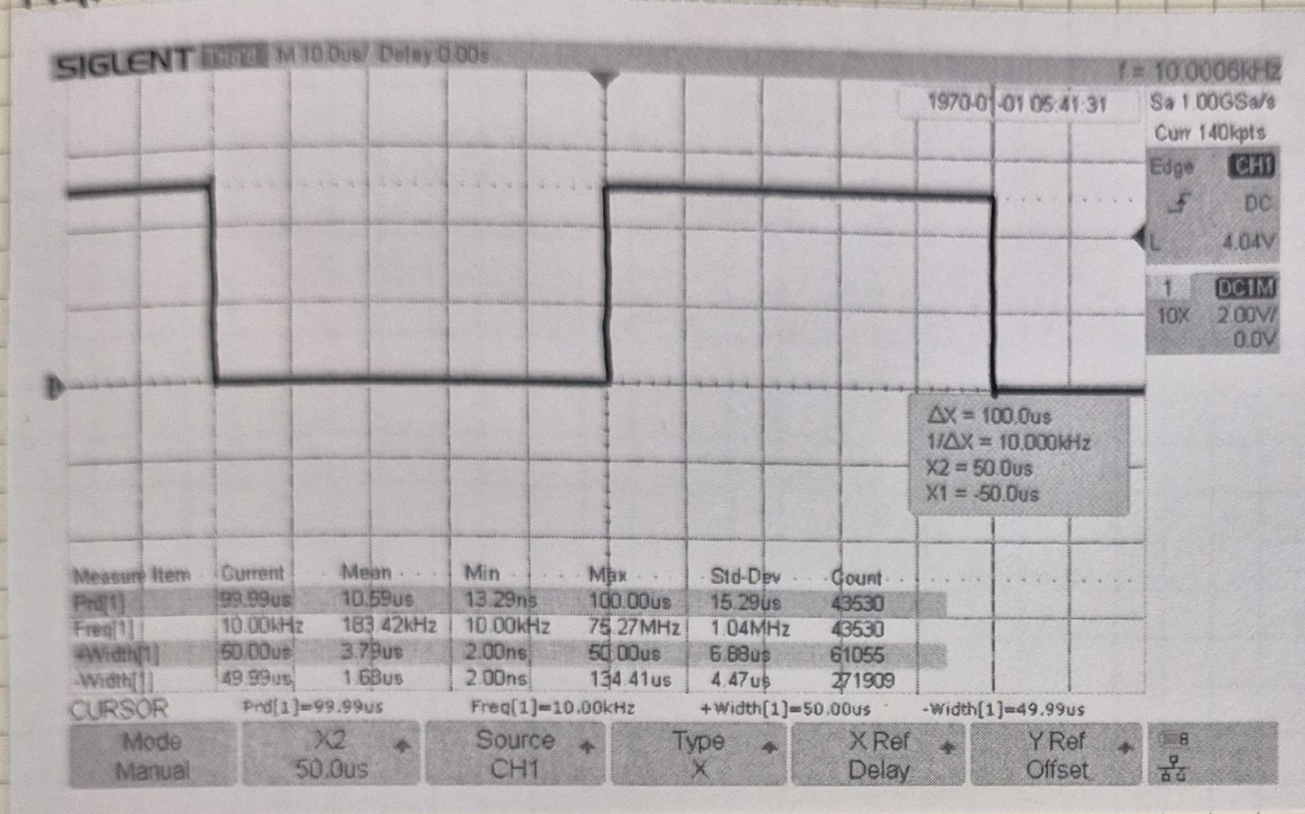


Table 3.2: Measured Delay Data

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	50 μ S	50 μ S	50 μ S	50.003 μ S
PS	50 μ S	50 μ S	50 μ S	49.993 μ S
Period	100 μ S	100 μ S	100 μ S	99.996 μ S
Frequency	10 KHz	10 KHz	10 KHz	10.000 KHz
Duty cycle	50%	50%	50%	50%

TJR
7-9-26

Experiment 3: Delays

Part 3: Create and verify a Long Delay

Goal:

Use nested software delay loops to create a much longer time interval that can be produced conveniently with a single 8-bit loop. Design a program that alternates the DLG7137 display between 5 and 0 with a nominal PW and PS of 0.5s. The timing calculation must be ~~appropriate~~^{highly} accurate to within one nominal instruction cycle of $\pm 1\mu\text{s}$. Then determine whether the available measurement equipment can verify that level of timing.

Before Lab:

Determine the longest delay that could be produced using the single 8-bit loop method from part 2 and explain why that method is not practical for creating a 0.5s delay.

One Loop Delay:

$$T_{\text{delay}} = 3N + 1$$

$$T_{\text{delay}} = 3(256) + 1$$

$$T_{\text{delay}} = 769\mu\text{s}$$

Design an inline nested delay loop capable of producing the required interval.

Do not use:

- Subroutines
- Time peripherals
- Interrupts

Account for every instruction executed between changes of the display value, including

- Initialization of each counter
- Inner-loop execution
- Outer-loop execution
- Skipped instructions
- branch instructions
- Instructions used to change the display value
- Any NOP instruction used for final timing adjustments

Use a nominal instruction-cycle time of $1\mu\text{s}$ to predict PW, PS, period, frequency, and duty cycle.

Experiment 3: Delays

Before Lab:

Figure 3.7: Long Delay Code

```

Main:
    MOVLW 0x05      ;1 TCY
    XORWF 0x07,1    ;1 TCY

    MOVLW 0x04      ;1 TCY
    MOVWF outCount  ;1 TCY
    goto OutDelay   ;2 TCY

OutDelay:
    MOVLW 0xC0      ;1 TCY
    MOVWF midCount  ;1 TCY
    goto MidDelay   ;2 TCY

MidDelay:
    MOVLW 0xD7      ;1 TCY
    MOVWF inCount   ;1 TCY
    goto InDelay    ;2 TCY

InDelay:
    DECFSE inCount  ;1 (2) TCY
    goto InDelay    ;2 TCY

    DECFSE midCount ;1 (2) TCY
    goto MidDelay   ;2 TCY

    DECFSE outCount ;1 (2) TCY
    goto OutDelay   ;2 TCY
    goto Main       ;2 TCY

End

```

$$\begin{aligned}
 &I3-1 \\
 &I3+3 \\
 &I3+6 \\
 &M(3I+6)+6 \\
 &O[M(3I+6)+6]-1 \\
 &O[M(3I+6)+6]+7
 \end{aligned}$$

$$\begin{aligned}
 O &= 4 \\
 M &= 192 \\
 I &= 215
 \end{aligned}$$

The reason for the numbers on O, M, and I is from choosing O and M, using those to solve for I. If each number needed to be solved it would require solving with mesh loops.

In the Lab:

Implement the calculated nested delay. The DLG7137 must alternate continuously between: $0 \rightarrow 5 \rightarrow 0 \rightarrow 5 \rightarrow \dots$ with a nominal 0.5s interval between changes. Show the modified code containing the nested delay.

Figure 3.8: Modified Long delay Code

```

Main:
    MOVLW 0x05      ;1 TCY
    XORWF 0x07,1    ;1 TCY

    MOVLW 0x04      ;1 TCY
    MOVWF outCount  ;1 TCY
    NOP
    NOP
    NOP
    goto OutDelay   ;2 TCY

OutDelay:
    MOVLW 0xC0      ;1 TCY
    MOVWF midCount  ;1 TCY
    NOP
    NOP
    NOP
    NOP
    goto MidDelay   ;2 TCY

MidDelay:
    MOVLW 0xD7      ;1 TCY
    MOVWF inCount   ;1 TCY
    goto InDelay    ;2 TCY

InDelay:
    DECFSE inCount  ;1 (2) TCY
    goto InDelay    ;2 TCY

    DECFSE midCount ;1 (2) TCY
    goto MidDelay   ;2 TCY

    DECFSE outCount ;1 (3) TCY
    goto OutDelay   ;2 TCY
    goto Main       ;2 TCY

```

NOPs were added because the exact timing isn't 1μs, using the NOPs allowed the code to produce a 0.5000004s delay. Oscilloscope capture and tables are located on the next page.

Experiment 3: Delays

In the Lab:

Figure 3.9: Delay oscilloscope capture

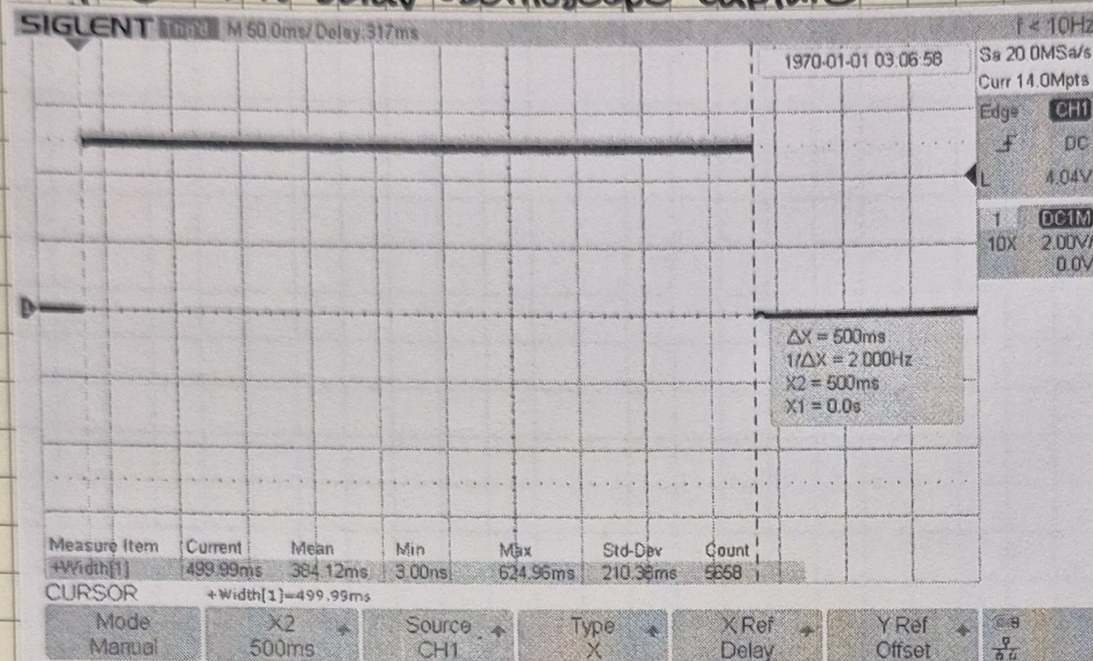


Table 3.3: Measured Long Delay Data

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	500 ms	500 ms	500ms	0.5000004 s
PS	500 ms	500 ms	500ms	0.5000006 s
Period	1000 ms	1000 ms	1000ms	1.0000007 s
Frequency	1 Hz	1 Hz	1 Hz	0.9999992 Hz
Duty cycle	50%	50%	50%	50.000 %

Table 3.4: NOP Table

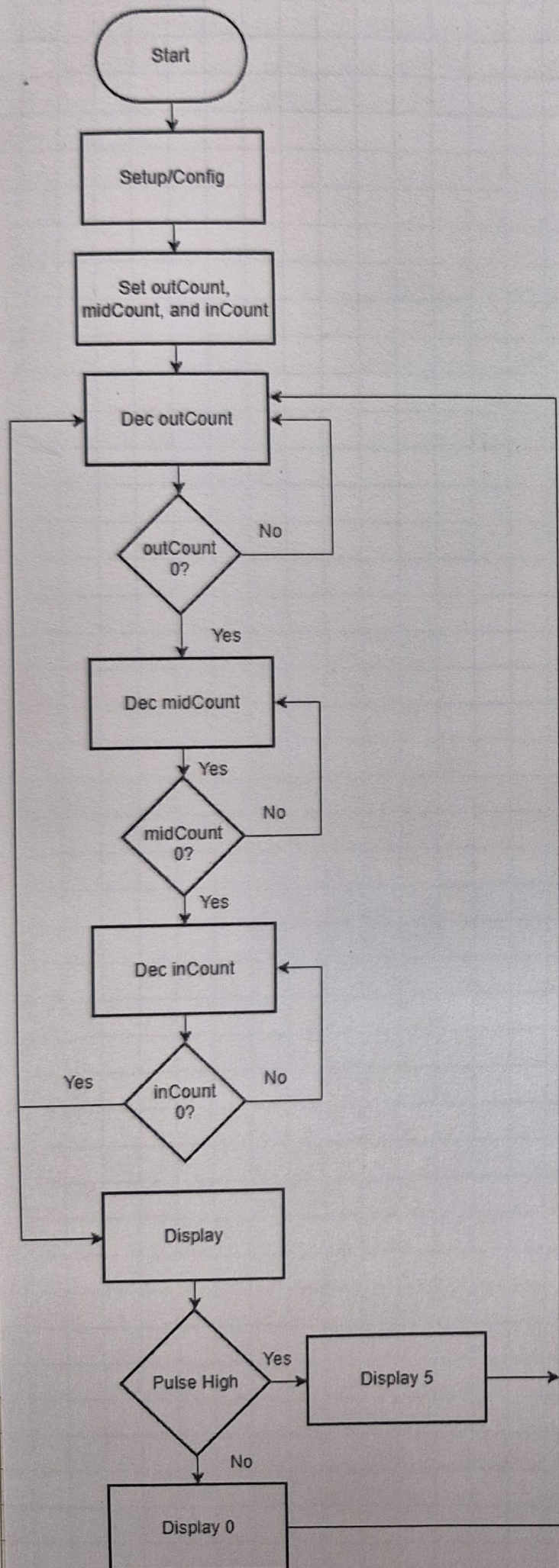
Measurement	Before NOP	With NOP	Difference	Instrument Resolution
PW	0.5000004 s	0.5000012 s	0.8 ns	100 pS Resolution

Measurements could not accurately be captured on the oscilloscope since the resolution isn't high enough to detect such small changes in the frequency and timing.

Experiment 3: Delay 5

In The Lab:

Figure 3.10: Long Delay Flowchart



TJR
9-10-26

Experiment 3: Delays

Part 4: Delay Subroutines and the Hardware Stack

Goal:

Refactor the working inline delay from Part 3 into reusable subroutines. Use `Call` and `Return`, including a nested subroutine call, and verify how the new program structure affects execution timing and the hardware return stack.

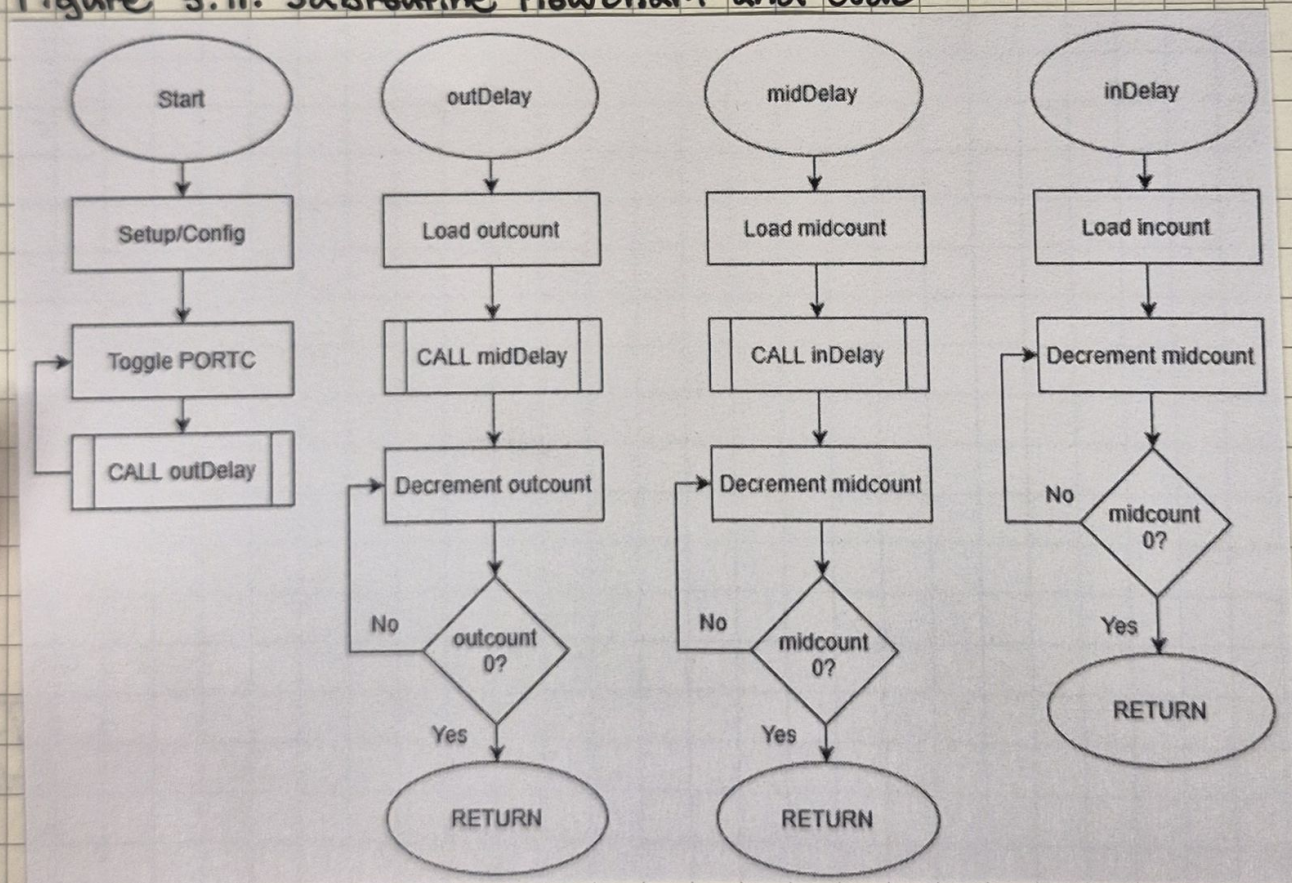
Before Lab:

Modify the Part 3 program so it contains:

- A main program that alternates the DLG7137 between 5 and 0
- A reusable long-delay subroutines
- A shorter delay subroutine called from within the long-delay subroutine
- Appropriate call and return instructions

Create and update the flowchart, showing the subroutine calls and return paths.

Figure 3.11: Subroutine Flowchart and Code



```
main:
    MOVW 0x05    : 1TCY
    XORWF 0x07,1 : 1TCY
    CALL outDelay : 2TCY
    NOP
    goto Main    : 2TCY

outDelay:
    MOVW 0x04    : 1TCY
    MOVWF outCount : 1TCY

outLoop:
    CALL midDelay : 2TCY
    DECFSS outCount : 1(2)TCY
    goto outLoop  : 2TCY
    RETURN        : 2TCY

midDelay:
    MOVW 0xB7    : 1TCY
    NOP
    NOP
    NOP
    NOP
    NOP
    MOVWF midCount : 1TCY

midLoop:
    CALL inDelay  : 2TCY
    DECFSS midCount : 1(2)TCY
    goto midLoop  : 2TCY
    RETURN        : 2TCY

inDelay:
    MOVW 0xE1    : 1TCY
    MOVWF inCount : 1TCY

inLoop:
    DECFSS inCount : 1(2)TCY
    goto inLoop    : 2TCY
    RETURN        : 2TCY

End
```

Recalculate the timing of the program.

$$O = 4 \quad M = 183 \quad I = 225$$

$$T_{\text{delay}} = O[M(3I+8)+8]+9$$

$$T_{\text{delay}} = 4 \cdot [183 \cdot (3 \cdot 225 + 8) + 8] + 9$$

$$T_{\text{delay}} = 499.997 \text{ ms}$$

3 NOPs can be added in the beginning of the code to get 500 ms.

Experiment 3: Delays

In the Lab:

1.) Implement the subroutines:

Build and verify the refactored code. Verify that the DLG7137 continuously alternates between $5 \rightarrow 0 \rightarrow 5 \rightarrow 0 \dots$

2.) Verify the timing:

Measure:

- PW
- PS
- Period
- Frequency
- Duty Cycle

Use the oscilloscope and frequency counter

Table 3.5: Delay Subroutine Measurement

Quantity	Predicted at 1 us/TCY	Expected Using Measured TCY	Oscilloscope	Frequency Counter
PW	500ms	500ms	500ms	0.5000001s
PS	500ms	500ms	500ms	0.5000001s
Period	1s	1s	1s	1.0000001s
Frequency	1Hz	1Hz	1Hz	0.9999998s
Duty cycle	50%	50%	50%	49.999%

Explain any difference between the inline implementation and the subroutine implementation.

There were not many differences between Part 3 and Part 4's results, the only changes were the needed NOPs for the measured 0.5s. For oscilloscope capture refer to page 32 since capture was the same results.

3.) Trace the nested calls:

Using the actual source code, trace one complete sequence:

```
Main
↑
→ CALL outlong Delay
  → CALL midDelay
    → CALL inDelay
      ← Return
    ← Return
  ← Return
Main continues Forever
```

The subroutine code is on the previous page for all code and instructions.

Experiment 3: Delays

Conclusion:

TJR
9-11-26

This lab demonstrates how instruction timing can be used to create and verify precise software delays with the PIC16F883. Instruction-cycle measurements were used to determine how individual instructions affect output timing, while DECFSZ loops were used to create both short and long delays. Nested loops allowed a 0.5 second delay to be produced for ~~attaching~~ alternating between 5 and 0 on the DLG7137, with the oscilloscope and frequency counter measurements used to compare calculated and actual timing. The delay was then converted into a reusable subroutine using CALL and RETURN, showing how nested subroutines calls affect execution time and use the hardware return stack. Overall, this lab showed the relationship between assembly instructions, instruction cycles, software delays, measured hardware timing, and subroutine operation.